Основе програмирања и програмски језици

1 Вежбе 3: Гранање

1.1 If гранање

Гранање (условно гранање) у програмском језику Python омогућава програму да доноси одлуке и извршава различите делове кода у зависности од тога да ли је неки услов тачан (True) или нетачан (False). Главне наредбе за гранање у Python-у су: **if**, **elif** и **else**. Наредба if се пише коришћењем кључне речи **if**. У оквиру Листинга 1 је дат пример једноставног гранања.

Листинг 1: Пример једноставног гранања

```
a = 33
b = 200

if b > a:
    print("b je vece od a")
```

У Листингу 1 користимо две променљиве, a и b , које су део if наредбе како би се проверило да ли је b веће од a. Пошто је a једнако 33, а b је 200, знамо да је 200 веће од 33, па програм исписује на екрану дату поруку.

Наредба if процењује услов (израз који као резултат даје True или False). Ако је услов тачан (True), блок кода унутар if наредбе се извршава. Ако је услов нетачан (False), тај блок кода се прескаче (видети Листинг 2).

Листинг 2: Прескакање наредбе

```
number = 15

if number > 0:
    print("Broj je pozitivan")
```

Python се ослања на увлачење кода (празан простор на почетку линије) како би дефинисао опсег (блок) у коду. Други програмски језици у ову сврху често користе витичасте заграде {}. If наредба без увлачења изазваће грешку у програму (видети Листинг 3).

Листинг 3: if наредба без увлачења

```
a = 33
b = 200

if b > a:
print("b je vece od a") # Ovde cemo dobiti gresku (IndentationError)
```

За увлачење можемо користити размаке (spaces) или табулаторе (tabs), али морамо користити исту количину размака за све наредбе унутар истог блока кода. Унутар једног if блока можемо имати више линија кода. Све те наредбе морају бити увучене на истом нивоу (видети Листинг 4).

Листинг 4: Више линија кода у једном гранању

```
age = 20

if age >= 18:
```

```
    print("Punoletni ste")
    print("Mozete da glasate")
    print("Mozete da otvorite bankovni racun")
```

Логичке променљиве (Boolean - True/False) могу се користити директно у if наредбама без оператора поређења (видети Листинг 5).

Листинг 5: Директно коришћење логичких променљивих

```
is_logged_in = True

if is_logged_in:
    print("Dobrodosli nazad!")
```

Python може да процени многе типове вредности као True или False унутар if наредбе. Нула (0), празни текстуални низови (), вредност None и празне колекције (листе, торке, сетови) третирају се као False. Све остало се третира као True. То укључује позитивне бројеве (5), негативне бројеве (-3) и било који текстуални низ који није празан (чак се и текст "False"третира као True зато што је то попуњен текстуални низ).

Кључна рцч **elif** је Python-ов начин да каже: "Ако претходни услови нису били тачни, онда покушај овај услов". Кључна реч elifнам омогућава да проверимо више израза и извршимо блок кода чим се један од услова покаже као тачан. У примеру датом у Лситингу 6 a је једнако b, тако да први услов није тачан, али је elif услов тачан, па програм исписује на екрану дату поруку.

Листинг 6: Наредба elif

```
a = 33
b = 33

if b > a:
    print("b je vece od a")
elif a == b:
    print("a i b su jednaki")
```

Можемо имати онолико elif наредби колико нам је потребно. Python ће проверавати сваки услов редом и извршити први који је тачан (видети Листинг 7). У примеру датом у Листингу 7 програм проверава сваки услов редом. Пошто је резултат (score) 75, програм исписује да је оцена C (зато што је то први услов који је тачан).

Листинг 7: Више наредби elif

```
score = 75

if score >= 90:
    print("Ocena: A")
elif score >= 80:
    print("Ocena: B")
elif score >= 70:
    print("Ocena: C")
elif score >= 60:
    print("Ocena: D")
```

Када користимо elif , Python процењује услове од врха ка дну. Чим пронађе услов који је тачан, он извршава тај блок кода и прескаче све преостале услове. Извршиће се само први тачан услов. Чак и ако је више услова тачно, Python се зауставља након што изврши први блок који се поклопио (видети Листинг 8).

Листинг 8: Више наредби elif

```
age = 25
```

```
2
3  if age < 13:
4      print("Vi ste dete")
5  elif age < 20:
6      print("Vi ste tinejdzer")
7  elif age < 65:
8      print("Vi ste odrasla osoba")
9  elif age >= 65:
10     print("Vi ste starija osoba")
```

Користити elif када имамо више међусобно искључивих услова које треба проверити. Ово је ефикасније од коришћења више засебних if наредби јер Python престаје са провером чим пронађе један тачан услов.

Листинг 9: Пример коришћења elif

```
1  day = 3
2
3  if day == 1:
4      print("Ponedeljak")
5  elif day == 2:
6      print("Utorak")
7  elif day == 3:
8      print("Sreda")
9  elif day == 4:
10     print("Cetvrtak")
11 elif day == 5:
12     print("Petak")
13 elif day == 6:
14     print("Subota")
15 elif day == 7:
16     print("Nedelja")
```

Кључна реч else хвата све што није обухваћено претходним условима. Наредба else се извршава када се испостави да су if услов (као и сви евентуални elif услови) нетачни (False). У Листингу 10 је дат пример где је a веће од b, тако да први услов није тачан, а није тачан ни elif услов. Због тога програм прелази на else услов и исписује на екрану дату поруку.

Листинг 10: Наредба else

```
1  a = 200
2  b = 33
3
4  if b > a:
5      print("b je vece od a")
6  elif a == b:
7      print("a i b su jednaki")
8  else:
9      print("a je vece od b")
```

Можемо користити else директно након if наредбе, без elif(видети Листинг 11). Ово ствара једноставан избор између две опције: ако је услов тачан, извршава се први блок, у супротном, извршава се else блок.

Листинг 11: if ... else

```
1  a = 200
2  b = 33
3
4  if b > a:
```

```
    print("b je vece od a")
else:
    print("b nije vece od a")
```

Наредба else обезбеђује подразумевану акцију када ниједан од претходних услова није тачан. Замислимо је као сигурносну мрежу која хвата сваки сценарио који нисмо покрили if и elif изјавама. Наредба else мора увек доћи на самом крају. Не можемо написати elif након else наредбе.

Листинг 12: Провера парности броја

```
number = 7

if number % 2 == 0:
    print("Broj je paran")
else:
    print("Broj je neparan")
```

Можемо комбиновати if, elif и else како бисмо креирали свеобухватну структуру за доношење одлука у коду.

Листинг 13: if ...elif ... else

```
temperature = 22

if temperature > 30:
    print("Napolju je vruce!")
elif temperature > 20:
    print("Napolju je toplo")
elif temperature > 10:
    print("Napolju je sveze")
else:
    print("Napolju je hladno!")
```

Наредба else служи као резервна варијанта која се покреће када закажу сви претходни услови. То је чини веома корисном за обраду грешака, валидацију података и постављање подразумеваних вредности (видети Листинг 14).

Листинг 14: Валидација корисничког уноса

```
username = "Milos"

if len(username) > 0:
    print(f"Dobrodosli, {username}!")
else:
    print("Greska: Korisnicko ime ne moze biti prazno")
```

Ако имамо само једну наредбу коју треба извршити, можемо је ставити у исти ред са if наредбом (видети Листинг 15). И даље нам је потребна двотачка : након услова.

Листинг 15: if наредба у једном реду

```
a = 5
b = 2

if a > b: print("a je vece od b")
```

Ако имамо једну наредбу за if и једну за else, можемо их ставити у исти ред користећи условни израз (видети Листинг 16). Ово се назива условни израз (conditional expression), а у програмирању је познат и као тернарни оператор.

Листинг 16: if...else наредбе у једном реду

```
a = 2
b = 330

print("A") if a > b else print("B")
```

Такође можемо користити ff / else у једном реду да бисмо изабрали вредност и доделили је променљивој(видети Листинг 17). Синтакса прати овај шаблон: promenljiva = vrednost_ako_je_tacno if uslov else vrednost_ako_je_netacno.

Листинг 17: if...else за доделу вредности

```
a = 10
b = 20

bigger = a if a > b else b
print("Veci broj je", bigger)
```

Можемо нанизати више условних израза у једном реду, али би то требало бити кратко како би код остао читљив (видети Листинг 18).

Листинг 18: Више услова у једном реду

```
a = 330
b = 330

print("A") if a > b else print("=") if a == b else print("B")
```

Скраћене if наредбе и тернарне операторе треба користити када су услов и акције једноставни,то побољшава читљивост кода, желимо да извршимо брзу доделу вредности на основу услова. Иако скраћени записи чине код краћим, треба избегавати њихову претерану употребу код слозхенијих услова. Зарад читљивости кода, користити обичне, вишелинијске if-else структуре када радимо са компликованом логиком.

Логички оператори се користе за комбиновање условних исказа. Кључна реч анд је логички оператор и користи се за комбиновање условних исказа. Оба услова морају бити тачна да би читав израз био тачан.

Листинг 19: Оператор and

```
a = 200
b = 33
c = 500

if a > b and c > a:
    print("Oba uslova su tacna (True)")
```

Кључна реч or је логички оператор и користи се за комбиновање условних исказа. Најмање један услов мора бити тачан да би читав израз био тачан.

Листинг 20: Оператор and

```
a = 200
b = 33
c = 500

if a > b or a > c:
    print("Barem jedan od uslova je tacan (True)")
```

Кључна реч not је логички оператор и користи се за преокретање (негацију) резултата условног исказа.

Листинг 21: Оператор not

```
a = 33
b = 200

if not a > b:
    print("a NIJE vece od b")
```

Можемо комбиновати више логичких оператора у једном изразу. Python прво процењује not, затим and, па на крају or.

Листинг 22: Комбиновање логичких оператора

```
age = 25
is_student = False
has_discount_code = True

if (age < 18 or age > 65) and not is_student or has_discount_code:
    print("Popust se primenjuje!")
```

Логички оператори у Python-у прате стандардна правила логике (таблице истинитости за and и or), где заграде служе за јасније дефинисање приоритета и услова у сложенијим изразима.

Можемо имати if исказе унутар других if исказа. То се назива угњеждени (nested) if искази.

Листинг 23: Угњеждени if

```
x = 41

if x > 10:
    print("Iznad deset,")
    if x > 20:
        print("a takodje i iznad 20!")
    else:
        print("ali ne i iznad 20.")
```

Сваки ниво угњежђавања креира дубљи ниво доношења одлука. Код се процењује од најудаљенијег (спољашњег) услова ка унутра.

Листинг 24: Угњеждени if

```
age = 25
has_license = True

if age >= 18:
    if has_license:
        print("Mozete da vozite")
    else:
        print("Potrebna vam je dozvola")
else:
    print("Mladi ste da biste vozili")
```

Можемо угњездити онолико нивоа колико нам је потребно, али треба имати на уму да превише нивоа може учинити код теже читљивим.

Листинг 25: Три нивоа угњежђавања

```
score = 85
attendance = 90
submitted = True

if score >= 60:
    if attendance >= 80:
```

```
        if submitted:
            print("Prolaz sa dobrim statusom")
        else:
            print("Prolaz, ali nedostaje zadatak")
    else:
        print("Prolaz, ali slaba prisutnost")
else:
    print("Pad")
```

Понекад се угњеждени if искази могу поједноставити коришћењем логичких оператора као што је and .

Листинг 26: Пример са логичким оператором

```
temperature = 25
is_sunny = True

if temperature > 20:
    if is_sunny:
        print("Savrseno vreme za plazu!")

temperature = 25
is_sunny = True

if temperature > 20 and is_sunny:
    print("Savrseno vreme za plazu!")
```

Искази if не могу бити празни. Ако из неког разлога имамо if исказ без икаквог садржаја, убацује се наредба pass како бисмо избегли грешку у коду. Наредба pass је такозвана "null" операција – када се изврши, не дешава се апсолутно ништа. Она служи искључиво као место за чување позиције (placeholder).

Листинг 27: pass

```
a = 33
b = 200

if b > a:
    pass
```

Корисна је када креирамо структуру кода али још нисмо имплементирали саму логику, када је исказ синтаксички обавезан а никаква акција није потребна, као ознака за будући код током развоја, или у празним функцијама и класама које планирамо да напишемо касније. Током програмирања, често ћемо желети да скицирамо структуру програма пре него што имплементирамо детаље. Наредба pass нам омогућава да то урадимо без појаве синтаксичких грешака. Python у потпуности игнорише коментаре, док је pass стварна наредба која се извршава (иако не ради ништа). Зато морамо користити pass на местима где Python захтева наредбу, јер сам коментар није довољан за блок кода.

Листинг 28: Примери pass

```
age = 20

if age < 18:
    pass  # TODO: Dodaj logiku za maloletnike kasnije
else:
    print("Pristup je odobren")


score = 85
```

```
10
11 if score > 90:
12     # Ovo je odlican rezultat
13     # PAZNJA: Ovo ce izazvati IndentationError gresku!
14
15
16 score = 85
17
18 if score > 90:
19     pass  # Ovo je odlican rezultat
20
21 print("Rezultat je obradjen")
22
23
24 value = 50
25
26 if value < 0:
27     print("Negativna vrednost")
28 elif value == 0:
29     pass  # Slucaj kada je nula - akcija nije potrebna
30 else:
31     print("Pozitivna vrednost")
32
33
34 def izracunaj_popust(cena):
35     pass  # TODO: Implementiraj logiku za popust kasnije
36     # Funkcija postoji, ali jos uvek ne radi nista
```

Задатак 1.

Написати програм којим се рачуна износ пореза на плату. Ако је плата већа од 1000 евра, порез је 20%, иначе је 15%.

Листинг 29: Решење Задатка 1.

```
1 plata = int(input())
2 if plata > 1000:
3    porez = plata * 0.2
4 else:
5    porez = plata * 0.15
6 print(porez)
```

Задатак 2.

Написати програм којим се, без употребе одређује највећи од три унета реална броја.

Листинг 30: Решење Задатка 2.

```
1 a = float(input())
2 b = float(input())
3 c = float(input())
4 m = a
5 if b > m:
6    m = b
7 if c > m:
8    m = c
9 print(m)
```

Задатак 3.

Написати програм који за унети цео број x израчунава вредност броја y, ако је:

$$y = \begin{cases} x^2, & x < -15 \\ x + 20, & -15 \leq x < 23 \\ x, & x \geq 23 \end{cases}$$

Листинг 31: Решење Задатка 3.

```
x = int(input("Unesite ceo broj x: "))
if x < -15:
   y = x**2
if (x >= -15) and (x < 23) :
   y = x + 20
if x >= 23 :
   y = x
print(y)
```

Задатак 4.

Написати програм којим се за три унета позитивна цела броја испитује да ли чине Питагорину тројку бројева.

Листинг 32: Решење Задатка 4.

```
a = int(input())
b = int(input())
c = int(input())
if (a*a==b*b+c*c) or (b*b==a*a+c*c) or (c*c==a*a+b*b):
   print("Uneti brojevi cine Pitagorinu trojku.")
else:
   print("Uneti brojevi ne cine Pitagorinu trojku.")
```

Задатак 5.

Написати програм који симулира мрежни firewall . Корисник уноси број порта (port), величину мрежног пакета у килобајтима (velicina_paketa) и протокол као текст (протокол - нпр.”TCP”, ”UDP”, ”ICMP”). Firewall пропушта пакет ако испуњава следећа правила: Ако је протокол ”ICMP”, пакет се пропушта само ако је величина пакета мања од 1 KB. У супротном се одбија; Ако је порт 80 или 443 (HTTP/HTTPS), пакет се увек пропушта, осим ако му је величина већа од 5000 KB ; За све остале портове важи правило: протокол мора бити ”TCP”и величина пакета не сме прелазити 1500 KB; У свим осталим случајевима пакет се одбија.

Листинг 33: Решење Задатка 5.

```
port = int(input("Unesite broj porta: "))
velicina_paketa = float(input("Unesite velicinu paketa (KB): "))
protokol = input("Unesite protokol (TCP/UDP/ICMP): ").upper()

if protokol == "ICMP":
    if velicina_paketa < 1.0:
        odluka = "PROPUSTEN"
    else:
```

```
        odluka = "ODBIJEN (ICMP paket prevelik)"
elif port == 80 or port == 443:
    if velicina_paketa <= 5000.0:
        odluka = "PROPUSTEN"
    else:
        odluka = "ODBIJEN (Web paket prevelik)"
else:
    if protokol == "TCP" and velicina_paketa <= 1500.0:
        odluka = "PROPUSTEN"
    else:
        odluka = "ODBIJEN (Nedozvoljen protokol/velicina za ovaj port)"

print(f"Odluka firewall-a: {odluka}")
```

Задатак 6.

Написати програм који процењује безбедност лозинке. Корисник уноси дужину лозинке (duzina), и три одговора (са ђа"или "не") на питања: да ли садржи велика слова (velika_slova), да ли садржи бројеве (brojevi) и да ли садржи специјалне карактере (specijalni). Ако је дужина мања од 6 карактера, лозинка је аутомацки "NEBEZBEDNA". Ако лозинка има 12 или више карактера и садржи сва три додатна елемента (велика слова, бројеве и специјалне карактере), оцена је "JAKO BEZBEDNA". Ако лозинка има бар 8 карактера и садржи бар два од три додатна елемента, оцена је "UMERENO BEZBEDNA". У свим осталим случајевима (нпр. има дужину али нема довољно елемената), оцена је ŠLABA".

Листинг 34: Решење Задатка 6.

```
duzina = int(input("Unesite duzinu lozinke: "))
has_velika = input("Da li sadrzi velika slova (da/ne): ").lower() == "da"
has_brojevi = input("Da li sadrzi brojeve (da/ne): ").lower() == "da"
has_specijalni = input("Da li sadrzi specijalne karaktere (da/ne): ").lower() == "da"

# Brojimo koliko je dodatnih uslova ispunjeno
brojac_elemenata = 0
if has_velika:
    brojac_elemenata += 1
if has_brojevi:
    brojac_elemenata += 1
if has_specijalni:
    brojac_elemenata += 1

if duzina < 6:
    status = "NEBEZBEDNA"
elif duzina >= 12 and brojac_elemenata == 3:
    status = "JAKO BEZBEDNA"
elif duzina >= 8 and brojac_elemenata >= 2:
    status = "UMERENO BEZBEDNA"
else:
    status = "SLABA"

print(f"Ocena lozinke: {status}")
```

Уместо писања многобројних if..else наредби, можемо користити наредбу match. Израз у match делу се процењује (евалуира) једном. Вредност израза се пореди са вредностима сваког case услова. Ако се вредности поклапају, извршава се припадајући блок кода (вдиети Листинг 35.

Листинг 35: Примери match

```
day = 4
match day:
  case 1:
    print("Monday")
  case 2:
    print("Tuesday")
  case 3:
    print("Wednesday")
  case 4:
    print("Thursday")
  case 5:
    print("Friday")
  case 6:
    print("Saturday")
  case 7:
    print("Sunday")
```

Користити карактер доње црте _ као последњу вредност у case услову уколико желимо да се тај блок кода изврши када нема других поклапања. Вредност _ ће се увек поклопити, па је због тога важно поставити је као последњи case како би се понашала као подразумевани случај (default).

Листинг 36: Подразумевани match

```
day = 4
match day:
  case 6:
    print("Today is Saturday")
  case 7:
    print("Today is Sunday")
  case _:
    print("Looking forward to the Weekend")
```

2 Самосталне вежбе 3

Задатак 1.

Написати програм који од корисника тражи да унесе број освојених поена на испиту (од 0 до 100). Програм треба да испише да ли је студент "Положио"(51 или више поена) или је "Пао"(мање од 51 поена).

Задатак 2.

Написати програм који учитава два цела броја. Ако је први број дељив другим бројем без остатка, програм исписује "Дељив је", у супротном исписује "Није дељив". (Осигурати се да други број није нула, у том случају исписати "Грешка: Дељење нулом").

Задатак 3.

Продавница рачунарске опреме даје попуст од 10% на сваку куповину чији је укупни износ већи од 5000 динара. Написати програм који учитава почетни износ рачуна, а затим израчунава и исписује коначну цену коју купац мора да плати.

Задатак 4.

Написати програм који симулира једноставан систем за праћење рада батерије на IoTуређају. Корисник уноси тренутни проценат батерије (baterijaод 0 до 100) и текстуални статус да ли је уређај прикључен на пуњач (punjac - đa"или "не"). Систем треба да испише обавештење према следећим правилима:

- Ако је проценат 100, а уређај је на пуњачу, исписати: "Батерија је пуна. Искључите пуњач ради очувања здравља батерије."
- Ако је батерија испод 20%, а уређај није на пуњачу, исписати: "Критично низак ниво батерије! Прелазак у Пощер Савинг Моде."
- Ако је батерија испод 20%, али је уређај на пуњачу, исписати: "Батерија се пуни (Низак ниво)."
- У свим осталим случајевима исписати: "Статус уређаја је стабилан."

Задатак 5.

Написати програм који за унети реални број x израчунава y на следећи начин:

$$y = \begin{cases} x, & x < 3 \\ x^2, & 3 \leq x < 5 \\ x^3, & \text{иначе} \end{cases}$$

Задатак 6.

Приликом преузимања великих фајлова преко заједничке мреже, менаџер преузимања додељује приоритет пропусности (протока) за два различита корисничка пакета ("Premium"и Štandard"). Написати програм који учитава тип пакета корисника и тренутно оптерећење мреже у MB по секунди (саобрацај у $Mbps$). Правила алокације брзине преузимања:

- Ако је укупни саобраћај на мрежи мањи од 50 Мбпс (мрежа је слободна), обе категорије добијају максималну брзину: "Додељен пун проток (Максимална брзина)."
- Ако је саобраћај између 50 и 150 Мбпс, "Премиум"корисници задржавају пуну брзину, док се Čтандард"корисницима ограничава проток:
  - За Премиум: "Преузимање: 100% брзине."
  - За Стандард: "Преузимање: 50% брзине (Ограничење због саобраћаја)."
- ко оптерећење мреже пређе 150 Мбпс (гужва), уводи се рестрикција за све:
  - Премиум добија: "Преузимање: 50% брзине."
  - Стандард добија: "Преузимање привремено паузирано (Ресурси преусмерени)."

3 Задаци за вежбање

Задатак 1.

Написати програм који уноси износ корпе купца. Ако је износ већи од 5.000 динара, купац добија попуст од 10% на целу корпу. Бесплатна достава се добија ако је коначни износ (након примењеног попуста) већи од 4.500 динара. У супротном, достава се наплаћује 300 динара. Програм треба да испише износ попуста, цену доставе и укупан рачун који купац мора да плати.

Задатак 2.

Компанија бира кандидате за IT позицију. Корисник уноси године радног искуства (цео број) и број програмских језика које познаје (цео број). Ако кандидат има мање од 2 године искуства, проверава се број језика:

- Ако зна 3 или више језика, добија статус Јуниор са потенцијалом.
- У супротном добија статус Практикант

Ако има између 2 и 5 година искуства (укључујући и те вредности), добија статус Медиор. Ако има више од 5 година искуства:

- Ако зна мање од 2 језика, добија статус Искусан специјалиста.
- Ако зна 2 или више језика, добија статус Сениор архитекта.

4 Литература/Корисни линкови

1. `https://imi.pmf.kg.ac.rs/moodle/course/view.php?id=327`

Датум ажурирања: 24.10.2026.